GRPO Reward Ablation for Arithmetic LLMs

Jiale Chen
jialechen2025@u.northwestern.edu
Track / Option: Engineer Track / Option 2 (Arithmetic LLM Optimization)

Abstract

This project studies how reward design affects GRPO training for an Arithmetic LLM. I compare three reward settings while keeping the model, tokenizer, prompt pool, verifier, and decoding configuration fixed. The first uses only a sparse binary exact-match reward. The second adds process and format rewards to provide denser supervision. The third adds a distance-based reward that gives partial credit to numerically close answers. Evaluation is based on exact-match arithmetic accuracy. All runs start from the same baseline accuracy of 0.1406. I find that both dense-reward variants improve early learning speed, reaching 0.30 accuracy by step 7, compared with step 23 for binary-only reward. However, late-stage behavior differs: binary + process/format reward underperforms with a final accuracy of 0.287, while binary + distance reward achieves the best final result at 0.342, slightly above binary-only at 0.340. The main conclusion is that denser rewards help only when they stay aligned with true task correctness.
GitHub Repository: *https://github.com/jiale0402/stat-359-final-proj*

1 Introduction

Reward design is a core issue in reinforcement-learning-based post-training. Sparse rewards are well aligned with task success, but they often make learning slow. Dense auxiliary rewards can improve optimization, but they may also encourage models to optimize a proxy instead of the real objective.

This project studies that trade-off on arithmetic reasoning, where correctness is easy to measure exactly. I compare three reward formulations for GRPO and ask:

- Do auxiliary rewards speed up early learning?
- Do they improve or hurt final exact-match accuracy?
- Which reward gives the best balance between learning speed and alignment?

My main contribution is a controlled reward ablation in which only the reward function changes across runs.

2 Background & Related Work

The course handout lists GRPO reward ablation as a key direction in the Arithmetic LLM optimization track. This project follows that goal directly by testing whether denser reward signals improve learning while preserving exact arithmetic correctness.

The core trade-off is simple. Binary exact-match reward is perfectly aligned with the evaluation metric, but it is sparse. Dense rewards provide more signal, but can drift away from what actually matters. Arithmetic is a useful testbed because the final answer is unambiguous, making it easy to detect reward misalignment.

3 Methodology & Experimental Setup

3.1 Data

The shared supervised checkpoint is built from:

- 200,000 synthetic arithmetic pretraining samples,
- 4,096 SFT instruction samples,
- a 90/10 split: 3,686 train and 410 validation examples.

For GRPO, I generate a prompt pool of 50,000 arithmetic prompts from random expression trees with depth at most 5 and numbers in the range 1–20. Invalid expressions are filtered out.

3.2 Model and Shared Setup

All experiments start from the same SFT checkpoint and keep the following fixed:

- model checkpoint,
- tokenizer,
- verifier,
- prompt distribution,
- decoding configuration,
- evaluation metric.

SFT configuration:

- 1 epoch,
- batch size 64,
- learning rate 5×10^{-5},
- 58 optimizer steps.

3.3 Reward Designs

I compare three reward formulations:

H1: Binary-only reward

$$R = 1.0 \times \text{binary}$$

H2: Binary + process + format reward

$$R = 0.8 \times \text{binary} + 0.15 \times \text{process} + 0.05 \times \text{format}$$

H3: Binary + distance reward

$$R = 0.9 \times \text{binary} + 0.1 \times \exp(-|err|/10)$$

H1 is sparse but perfectly aligned with exact correctness. H2 adds denser supervision through reasoning-process and formatting rewards. H3 instead adds partial credit for answers numerically close to the true answer.

3.4 Evaluation Metrics

The main metric is **exact-match arithmetic accuracy**. I also compare:

- the step at which each run reaches 0.30 accuracy,
- final validation accuracy,
- auxiliary reward trends for diagnosing proxy optimization.

All runs share a step-0 baseline accuracy of 0.1406. H1 also serves as the main baseline because it directly optimizes the true task objective.

4 Results

Table 1: Comparison of reward designs.

Reward Design	Step to 0.30	Final Acc.
H1: Binary only	23	0.340
H2: Binary + process + format	7	0.287
H3: Binary + distance	7	**0.342**

Figure 1 shows the main training trajectories. Both H2 and H3 improve early learning substantially, reaching 0.30 accuracy by step 7 instead of step 23 for H1. This confirms that denser rewards can make optimization easier in the first part of training.

However, late-stage behavior reveals an important difference. H2 loses alignment after its fast

start: it peaks near 0.309 around step 11 and then declines to 0.287 by the end. H1 climbs more slowly but steadily, finishing at 0.340. H3 combines both benefits: it learns quickly, remains stable, and ends at the best final accuracy, 0.342.

4.1 Qualitative Findings

The reward-component plots make the difference between H2 and H3 much clearer. In H2, the proxy rewards improve strongly throughout training, but exact-match accuracy stalls and then drifts downward. Process reward rises from 0.59 to 0.89 and format reward rises from 0.73 to 1.00, yet the model does not become more correct on the arithmetic task. Figure 2 is strong evidence of proxy optimization: the model is learning to satisfy the easier reward components instead of the real objective.

In H3, the dense reward stays tied to answer quality. The distance reward rises together with exact-match accuracy and then levels off in a similar region, as shown in Figure 3. That alignment likely explains why H3 improves early learning without sacrificing final correctness.

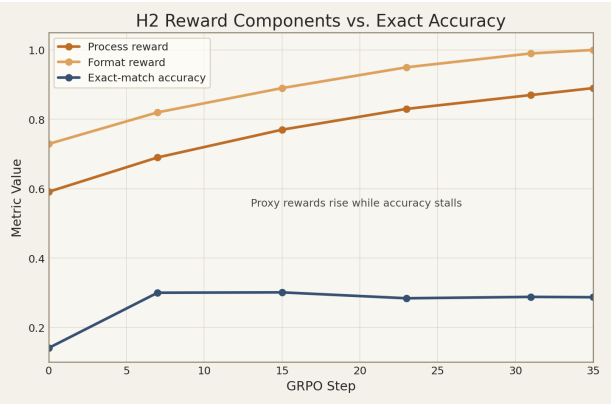

Figure 2: H2 reward components versus exact-match accuracy. Proxy rewards keep rising even while exact correctness stalls, indicating misalignment.

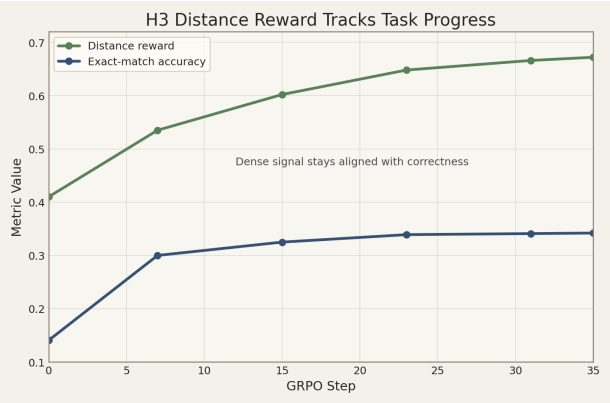

Figure 3: H3 distance reward versus exact-match accuracy. The dense reward tracks genuine task progress much more closely than H2's proxy components.

5 Discussion & Error Analysis

The main lesson is that not all dense rewards are equally useful. H1 is slow but reliable because it is fully aligned with exact correctness. H2 speeds up learning early, but its process and format terms encourage behaviors that are easier to optimize than actual arithmetic success. H3 works better because its additional signal remains connected to the real goal.

There are still important limitations. This study uses one model family, one task type, and only three reward formulations. The gap between H1 and H3 is also small in absolute terms, so the findings should be interpreted carefully. Reward design helps, but it does not solve arithmetic reasoning by itself.

6 Conclusion

This project evaluates how reward design affects GRPO training for an Arithmetic LLM. Binary-only reward gives the cleanest objective but suffers from sparse feedback. Adding process and format rewards improves early learning speed but reduces final exact-match accuracy, showing clear reward misalignment. Adding a distance-based reward provides the best balance: faster learning and the strongest final result.

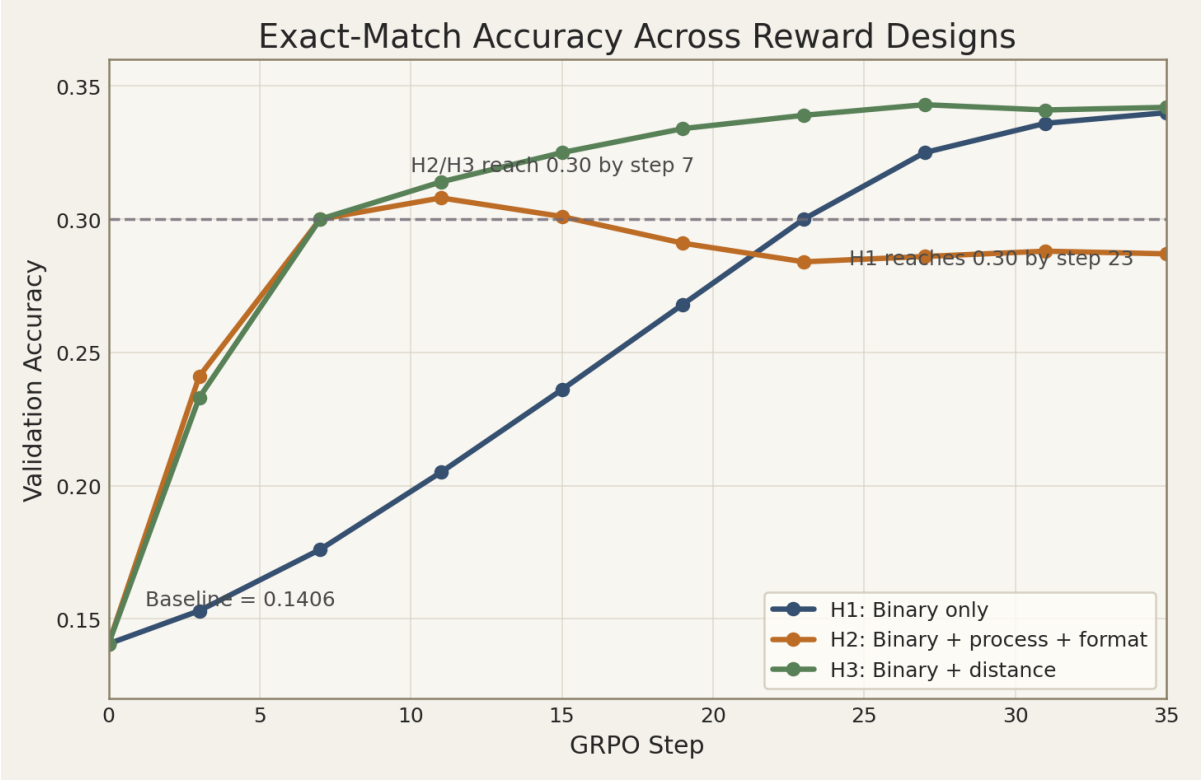

Figure 1: Validation exact-match accuracy across GRPO steps for the three reward designs. H2 and H3 hit 0.30 by step 7, while H1 reaches that threshold at step 23. H3 achieves the strongest overall trajectory at the end of training.

The main takeaway is that auxiliary rewards should reflect real progress toward the final task, not just surface-level structure. In future LLM training setups, I would prioritize dense rewards that preserve alignment with the true evaluation metric.

7 Individual Contributions

- **Jiale Chen:** Experimental design, training setup, evaluation, visualizations, slide preparation, and report writing.

8 References

References

[1] Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv preprint arXiv:2305.07759*.

[2] Northwestern University. Final Project: LLM Engineering & Prompting — Pretraining, Fine-Tuning, Data Design, Evaluation. Course handout.

A Optional Appendix

A.1 Key Experimental Settings

- Pretraining data: 200,000 synthetic arithmetic samples
- SFT data: 4,096 instruction samples
- RL prompt pool: 50,000 arithmetic prompts
- Prompt generation: depth ≤ 5, numbers 1–20
- Decoding: max length 256, temperature 1.0, top-k 50, top-p 0.95
- Primary metric: exact-match arithmetic accuracy

A.2 Reward Definitions

- H1: $R = 1.0 \times \text{binary}$
- H2: $R = 0.8 \times \text{binary} + 0.15 \times \text{process} + 0.05 \times \text{format}$
- H3: $R = 0.9 \times \text{binary} + 0.1 \times \exp(-|err|/10)$